

2-3. Tasks and Threads

Program execution have 3 parts

1. Decomposition
2. Assignment
3. Orchestration
4. Mapping

C++11 Multithreading

- Thread-aware memory model
- `main()` is the main thread
- When main thread attempts to return 0
 - All threads that are `joinable()` are terminated
 - If these threads are incomplete and interrupted, a **non-zero return code is produced**

Basic thread usage

```
#include <thread>

void hello(); //Some declared method to run in the thread

int main()
{
    std::thread t(hello);
    t.join(); //Wait to execute finish
}
```

```
// Using function object
class background_task
{
public:
    void operator()() const //Declare () operator to create function object
    {
        do_something();
    }
}

background_task f;
std::thread my_thread(f);

std::thread another_thread((background_task()));
std::thread another_thread2{background_task()}; //Recommended way to declare threads and all
initialization

// Using lambda
std::thread my_thread([]{
    do_something();
});
```

Waiting and detaching

```
t.join(); //Wait
t.joinable(); //Check if thread can be joined
```

```

t.detach(); //Main thread no need to wait; but caution local variables passed

/*-----Example 1-----*/
std::thread t(my_func);
try
{
    do_something_in_current();
}
catch(...)
{
    t.join(); //Thread MUST be joined even when there is exception
    throw;
}
t.join();

/*-----Example 2-----*/
t.detach();
assert(!t.joinable()); //Cannot be joined because t is already detached

```

Potential issues with detach

Ensure that if we pass local variables to the thread, ensure that the local variables are not destroyed while still being used

Passing argument to threads

```

void f(int i, std::string const& s);
std::thread t(f,3,"hello");

/*-----Example 2-----*/
/*
Potential issue with race condition
1. While creating thread t, detach thread t
2. Buffer released before complete thread creation
3. (Fail) Convert buffer to std::string fails
To fix, pass as std::string before thread creation
*/

void f(int i, std::string const& s);
char buffer[1024];
sprintf(buffer, "%i", 100);

std::thread t(f,3,buffer); //Potential issue
std::thread t(f,3,std::string(buffer));

t.detach();

```

Important Principles

Owners

- An object that contains a pointer to another object
 - This second object requires a `delete`
- Have 1 owner per object, do not have 0 owners either
- Ownership is not enforced by C++

Resource Acquisition is Initialization (RAII)

- Whoever creates the object must be the owner
- The owner has to pass the ownership to another before destructing or returning

- `std::thread` are movable but not copyable - you cannot have 2 owners of a thread

• Lines 11-19: Transfer ownership of thread out of function

```
11 std::thread f() {
12     void some_function();
13     return std::thread(some_function);
14 }
15 std::thread g() {
16     void some_other_function(int);
17     std::thread t(some_other_function, 42);
18     return t;
19 }
```

• Lines 21-28: Transfer ownership of thread into a function

```
21 void f(std::thread t);
22 void g()
23 {
24     void some_function();
25     f(std::thread(some_function));
26     std::thread t(some_function);
27     f(std::move(t));
28 }
```

Data Race

- Ensure interleaving of code does not cause any data races
- To avoid, use locks, ordered atomics and transactional memory

Monitors

- Condition variables don't have a "condition"
- Monitors prevent threads from concurrently accessing a shared object state and wait for state to change
- Use both mutex and condition variable

```
std::condition_variable cond;

void blocking()
{
    std::unique_lock lock{mutex};
    while(jobs.empty()) cond.wait(lock); //While waiting, the lock is released
}
```

4. Atomics and Memory model in Modern C++

*Atomic<> Weapons**

As-If Rule

Compilers can make any changes as long as

- Access (read write) to volatile objects occur strictly according to the semantics of the expressions in which they occur
- At program termination, data written to files is exactly as if program was executed as written
- Prompting text sent to interactive devices shown before program wait for input

Avoiding Data Races

Use

1. Locks (Lecture 3)
2. Ordered Atomics (This lecture)

Memory Model

Knowing the memory model allows us to know the **modification order (MO)**

Composed of writes to an object by multiple threads

- MO varies between runs
- For every object in the program
- Atomic operations used to enforce MO
- If the object is not one of the atomic types, programmer is responsible for ensuring threads agree on MO
- If atomic variable used, compiler responsible for ensuring necessary synchronization to ensure all threads agree on MO (`!= race_free`)

Memory Order using Atomics

`std::memory_order` enumeration

- **Default** ordering, the strongest ordering: `std::memory_order_seq_cst`
- **Store** operations can have `memory_order_relaxed`, `memory_order_release`, or `memory_order_seq_cst` ordering
- **Load** operations can have `memory_order_relaxed`, `memory_order_consume`, `memory_order_acquire`, or `memory_order_seq_cst` ordering
- **Read-modify-write** operations can have `memory_order_relaxed`, `memory_order_consume`, `memory_order_acquire`, `memory_order_release`, `memory_order_acq_rel`, or `memory_order_seq_cst` ordering

Modification Order Enforcement

Sequenced-before (`sb`)

- Within same thread, evaluation A may be *sequenced-before* evaluation B
- i.e. A is completed before B begins
- Synchronizes-with (`sw`)
- Between atomic load and store operations
- A store to an atomic variable while a load to the same variable on a different thread
- Simply happens-before (`hb`)
- Regardless of threads evaluation A simply happens-before evaluation B if any of the following is true
 1. A is sequenced-before B
 2. A synchronizes-with B
 3. A simply happens-before X and X simply happens-before B (transitive)Strongly happens-before
- Same as `hb`, but in all contexts
- Change to `hb` transitivity: A is `sb` X, X `hb` Y, and Y `sb` B
- Strongly happens-before is transitive

Guarantees on Atomic Variables

Each atomic variable has a modification order

- Write-write coherence
- Read-read coherence
- Read-write coherence
- Write-read coherence

1. Sequential Consistent Ordering

- All threads must agree on the same order of operations
- Should be the **default**
- Noticeable performance penalty on *weakly-ordered* machine
- You can have different interleaving of different runs
- Additional machine instructions used to keep synchronization

```
std::atomic<bool> x,y;  
  
x.store(true, std::memory_order_seq_cst);  
  
x.load(std::memory_order_seq_cst)
```

2. Non-Sequentially Consistent Ordering

- **Do NOT think about how instructions are ordered/interleaved**
- All that matters is all threads agree on **modification order for each individual variable**

2.1 Relaxed Ordering

- Guarantee atomicity and modification order consistency
- Do not have `sw` relationship

```
x.store(true, std::memory_order_relaxed);  
  
x.load(std::memory_order_relaxed)
```

2.2 Release-Acquire Ordering

- For a specific variable, a `store` with `memory_order_release` and a `load` with `memory_order_acquire` has a `sw` relationship

```
x.store(true, std::memory_order_release);  
  
x.load(std::memory_order_acquire)
```

5. Concurrent Data Structures

Invariants

- These are usually broken **during an update** but maintained after each operation
- e.g. When updating a LinkedList, the pointers may be broken while a delete is taking place

Concurrency while calling Functions

- Constructors and destructors: Requires exclusive access to data structure
- Data structure that support assignment (=), swap, copy construction:
 - See if operations are thread-safe
- Identify the smallest region of serialized code

Nonblocking Data Structures

A confusing explanation:

1. Obstruction-free
2. Lock-free

3. Wait-free

Lock-Free Algorithms

Wikipedia Definition

- Allow individual threads to starve but system-wide throughput
- One of the threads make progress

Lock-Free Data Structures

- More than one thread must be able to make progress
- Progress can be defined as
 - Completed if a function returns
 - Completed if effects are **visible** to other threads

Wait-Free Data Structures

- Lock-free algorithms with loops (using compare/exchange operations) can result in one thread starving
- Algorithms may involve unbounded number of retries
- Wait-free data structures are extremely hard; You may end up writing a spin lock

Pros of Lock-Free Data Structures

- Maximum concurrency
- Robustness: If a thread dies halfway, nothing is lost except that thread's data; We cannot exclude threads from accessing the data structure

Cons of Lock-Free Data Structures

- Livelocks are possible
- Decreases overall performance
 - Even though they reduce time waiting, there's high memory contention and write propagation
 - Atomic operations used for lock-free code can be much slower than non-atomic operations
- False sharing

Writing Lock-Free Code

- Use `std::memory_order_seq_cst` for prototyping
- Use lock-free memory reclamation scheme
 - Recycle nodes for data structure
- ABA problem
- Identify busy-wait loops

6. Testing & Debugging

Concurrency-related bugs

- Unwanted blocking
- Race conditions

Unwanted Blocking

- Deadlock/livelock
 - No progress is made when it should be possible
 - Progress: Operation gets completed - effects become visible
- I/O or external input blocks

Issues

- Data races
 - Observable due to unsynchronized concurrent access to shared memory location
- Broken invariants
 - Dangling pointers, Random memory corruption, Double free
- Lifetime issues
 - Use-after-free, out of scope
 - Can be mitigated by using `join()` but also must remember to handle thrown exception

Designing for Testability

- Responsibility of each function and class are clear
- Functions are short and to the point
- Tests can take complete control of environment surrounding the code being tested
- Code that performs similar operation is tested close together

Debugging Tools

- Tools keep track of memory state
 - Computes happen-before to find data race

Valgrind

- Heavy weight binary instrumentation (20x overhead)
- Shadows all program values
- Creates shadow memory - Tracks and store information on memory used by program and detects incorrect access to memory
- Implementation
 - A-Bit: Valid address, check if byte is accessible
 - V-Bit: Valid Value, check if bit is initialized
- Helgrind
 - Additional dynamic instrumentation
 - Intercepts calls to functions and instruments
 - 100x overhead
 - Detects: Misuses of POSIX pthreads API, potential deadlocks arising from lock ordering and data races
 - Builds directed graph to determine order of locks
 - When thread acquires new lock,
 - Graph updated and check for cycles
 - Presence of cycle => Potential deadlock
 - Build directed acyclic graph to check happens-before relationships to monitor order in memory access

```
valgrind --tool=memcheck <prog ...>
```

Sanitizers

- Compilation-based approach
- Lighter weight
- 2-5x overhead
- Examples: `AddressSanitizer`, `LeakSanitizer`, `ThreadSanitizer` (TSan), `MemorySanitizer`, `HWASAN` (Hardware-assisted `AddressSanitizer`), `UBSan` (`UndefinedBehaviorSanitizer`)

```
g++ a.cpp -fsanitize=thread
g++ a.cpp -fsanitize=address
```

Address Sanitizer

- 2x slowdown
- 1.5-3x memory overhead
- Any aligned 8 bytes may have 9 states

Thread Sanitizer

- `-fsanitizer=thread`, `-fPIE`, `-pie`
- Intercepts function entry/exit and memory accesses
- Shadow Mapping
 - 8-byte shadow cell represents memory access
 - Race identified with there's no happen-before relationship for a specific operation and declare as a race

An 8-byte shadow cell represents one memory access:

- ~16 bits: TID (thread ID)
- ~42 bits: Epoch (scalar clock)
- 5 bits: position/size in 8-byte word
- 1 bit: IsWrite

Full information (no more dereferences)

TID

Epo

Pos

IsW

7. Go Lang

Sync Package

- `WaitGroup`
- Synchronization Primitives: `Mutex`, `RWMutex`, `Cond`, `Once`
- Constructs: `Pool`

```
var wg sync.WaitGroup
wg.Add(2)

defer wg.Done()
defer wg.Done()
```

Memory Model

- Data-race-free programs are **sequentially consistent**
- Happens Before: Sequenced Before or Synchronized Before
- Synchronized Before `sb` :

- `go` starts a new goroutine **synchronized before** the goroutine's execution begins
- `exit` of a goroutine is **not** guaranteed to be **synchronized before** any event
- `send` on a channel is **synchronized before** the completion of the corresponding receive
- `close` of a channel is **synchronized before** a receive that returns zero because it is closed
- `receive` on unbuffered channel is **synchronized before** completion of corresponding send on channel
- k^{th} receive on channel with capacity C is **synchronized before** $(k + c)^{th}$ send from channel completes
- TLDR of `sb`: Send and receive are synchronized with each other

8. Go Patterns

Confinement

- To achieve safe operation (e.g. `sync.Mutex` or channels)
- Types:
 - Ad-hoc confinement
 - Data only modified by one goroutine
 - Needs static analysis to ensure safety
 - Lexical
 - Restrict access to shared locations

Lexical Confinement

```
/*----- Example 1: Read/write split -----*/
// Can write to channel
chanOwner := func() <-chan int {
}
// Can only read from channel as it only receives channel head
consumer := func(results <-chan int) {
}

/*----- Example 2: Expose slice of array -----*/
data := []byte("golang")
go printData(&wg, data[:3])
go printData(&wg, data[3:])
```

for-select Loop

```
sum := 0
for { // Infinitely
    select { //Block until any event in select occurs
        case num := <-q:
            sum += num
        case <-done:
            sumCh <- sum
            return
    }
}
```

Done Channel

Leaking goroutines

- e.g. Waits infinitely for nil goroutine
- General Principle: It is responsibility of the goroutine creator to ensure it stops to prevent leaking

```
doWork := func(done <-chan interface{}, strings <-chan string) <-chan interface{} {
    terminated := make(chan interface{})
```

```

go func() {
    defer close(terminated)
    for {
        select {
            case s := <-strings:
                // Something
            case <-done:
                return
        }
    }
}()
return terminated
}

done := make(chan interface{})
terminated := doWork(done, nil)
// 1s timeout
go func() {
    time.Sleep(1 * time.Second)
    close(done)
}()

```

Error Handling

```

type Result struct {
    Error error
    Response *http.Response
}

```

Pipeline

- Possible to use fan-out to decrease processing time for specific page
- Not obviously faster than a task pool
 - Requires tweaking to make pipeline faster
- Better when a specific resource is needed by all tasks (e.g. I/O)
- Connect stages using channels

```

pipeline := multiply(done, add(done, multiply(done, insStream, 2), 1), 2)
for v := range pipeline {
    fmt.Println(v)
}

multiply := func (
    done <-chan interface{},
    intStream <-chan int,
    multiplier int,
) <-chan int {
    multipliedStream := make(chan int)
    go func() {
        defer close(multipliedStream)
        for i := range intStream {
            select {
            case <-done:
                return
            case multipliedStream <- i*multiplier:
            }
        }
    }
}

```

```
    return multipliedStream
}
```

Fan-Out, Fan-In

Fan-Out

```
numFinders := runtime.NumCPU() // Find out number of OS threads
finders := make([]chan int, numFinders)
for i := 0; i < numFinders; i++ {
    finders[i] = primeFinder(done, randIntStream)
}
```

Fan-In

```
var wg sync.WaitGroup
multiplexedStream := make(chan interface{})
multiplex := func(c chan interface{}) {
    defer wg.Done()
    for i := range c {
        select {
            case <-done:
                return
            case multiplexedStream <- i:
        }
    }
}
wg.Add(len(channels))

for _, c := range channels {
    go multiplex(c)
}

go func() {
    wg.Wait()
    close(multiplexedStream)
}()
return multiplexedStream
```

Load Balancer

Requesters

```
func requester(work chan<- Request) {
    c := make(chan int)
    for {
        Sleep(rand.Int63n(nWorker * 2 * second)) // Create request every 2s
        work <- Request{workFn, c}
        result := <-c
        furtherProcess(result)
    }
}
```

Workers

```
type Worker struct {
    requests chan Request // Work to do
    pending int //Number of pending tasks
    index int
}
```

```

}

func (w *Worker) work(done chan *Worker) {
    for {
        req := <-w.requests // Get request from balancer
        req.c <- req.fn() // Call function, send result
        done <- w // Send to done channel to let balancer know done
    }
}

```

Balancer

```

type Pool []*Worker

type Balancer struct {
    pool Pool
    done chan *Worker
}

func (b *Balancer) balance(work chan Request) {
    for {
        select {
        case req := <-work:
            b.dispatch(req)
        case w := <-b.done:
            b.completed(w)
        }
    }
}

// Read a worker from the heap and dispatch job to it, then add back to heap
func (b *Balancer) dispatch(req Request) {
    w := heap.Pop(&b.pool).(*Worker)
    w.requests <- req
    w.pending++
    heap.Push(&b.pool, w)
}

// Job done, update heap
func (b *Balancer) completed(w *Worker) {
    w.pending--
    heap.Remove(&b.pool, w.index)
    heap.Push(&b.pool, w)
}

```